

UNIT V

TREES AND GRAPHS

Trees: Introduction to Trees, Binary Search Tree – Insertion, Deletion & Traversal, AVL Trees

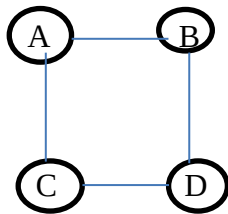
Graphs: Definition – Representation of Graph – Types of graph - Breadth-first traversal - Depth-first traversal – Applications of graphs.

GRAPH:

- A **graph** is a non-linear data structure used to represent relationships between elements.
- A graph is defined as $G=(V,E)$ consists of set of vertices V and set of Edges E .
- Vertices are referred to as nodes and connection between the nodes are referred as Edges

Example :

Let's take a simple graph:



- **Vertices (V)** = {A, B, C, D}
- **Edges (E)** = {(A, B), (A, C), (B, D), (C, D)}

This means:

- A is connected to B and C
- B is connected to D
- C is connected to D

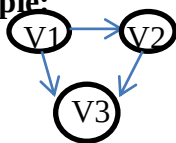
TYPES OF GRAPHS

1. Directed Graph (Digraph)

A graph in which edges have a **direction**.

- Represented as $(u \rightarrow v)$
- Connection is **one-way**

Example:



$(v1,v2) \neq (v2,v1)$

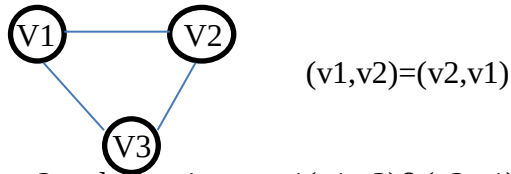
Here $v1$ points to $v2$ only $(v1,v2)$, $v2$ points to $v3$ only $(v2,v3)$, $v3$ points to $v1$ only $(v3,v1)$

2. Undirected Graph

A graph in which edges have **no direction**.

- Represented as (u, v)
- Connection is **bidirectional**

Example:



Here $v1$ points to $v2$ and $v2$ points to $v1$ ($v1,v2$) & ($v2,v1$)
 $(v2,v3)$ & ($v3,v2$)
 $(v3,v1)$ & ($v1,v3$)

3. Weighted Graph

A graph in which edges have **weights or costs**.

- Weights may represent distance, time, or cost.
- Weighted graph can be directed or undirected graph.

Example:

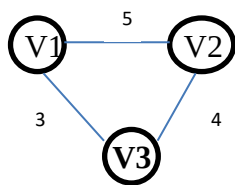


Fig 3.1: Undirected weighted graph

$(v1, v2, 5) \rightarrow$ edge from $v1$ to $v2$ and $v2$ to $v1$ with weight 5

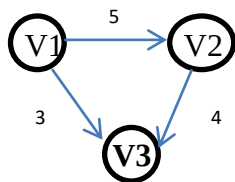


Fig 3.2: Directed weighted graph

$(v1,v2) \rightarrow$ edge from $v1$ to $v2$ with weight 5

4. Simple Graph

A graph with:

- No self-loops
- No multiple edges between same vertices

Example:



There are no loops between $v1$ and $v2$

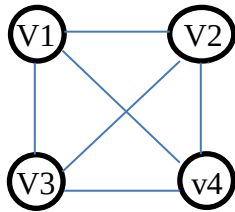
There is only one edge between any two nodes

5. Complete Graph

A graph in which **every vertex is connected to every other vertex**.

- If there are n vertices, edges = $n(n-1)/2$

Example:



Number of vertices:4

Number of Edges:6

6.Strongly connected Graph:

A directed graph is said to be strongly connected if there is a path from every vertex to every other vertex in the graph.

That means:

- For any two vertices **u** and **v**
- You can reach **v from u** and also **u from v**

Example:

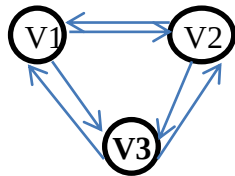


Fig : Strongly connected graph

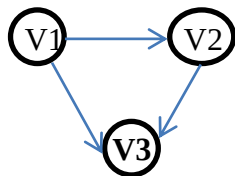


Fig: Weakly connected graph

REPRESENTATION OF GRAPH

Graphs can be represented in different ways:

1. Adjacency matrix
2. Adjacency list
3. Incidence Matrix
4. Edge List

1. Adjacency matrix:

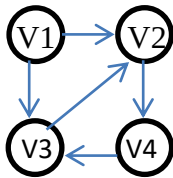
An **adjacency matrix** is a square matrix of size $n \times n$, where n is the number of vertices.

- If there is an edge between vertex i and $j \rightarrow$ value is **1** (or weight)
- If there is no edge $\rightarrow$ value is **0**
- An **adjacency matrix** is a way to represent a graph using a **2D array (matrix)**.

Example:

i) Adjacency matrix for directed graph

A 2D array of size 4×4

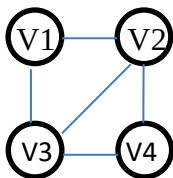


- **1** $\rightarrow$ edge exists
- **0** $\rightarrow$ no edge

	V1	V2	V3	V4
V1	0	1	1	0
V2	0	0	0	1
V3	0	1	0	0
V4	0	0	1	0

ii) Adjacency matrix for Undirected graph

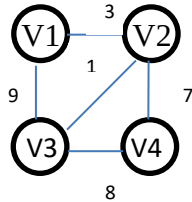
A 2D array of size 4×4



- **1** $\rightarrow$ edge exists
- **0** $\rightarrow$ no edge

	V1	V2	V3	V4
V1	0	1	1	0
V2	1	0	1	1
V3	1	1	0	1
V4	0	1	1	0

iii) Adjacency matrix for Weighted graph



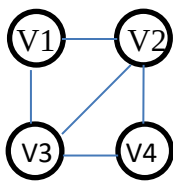
- 1 → edge exists, if there is no edge between v_i to $v_j \rightarrow \infty$
- 0 → no edge

	V1	V2	V3	V4
V1	0	3	9	$\infty/0$
V2	3	0	1	7
V3	9	1	0	8
V4	$\infty/0$	7	8	0

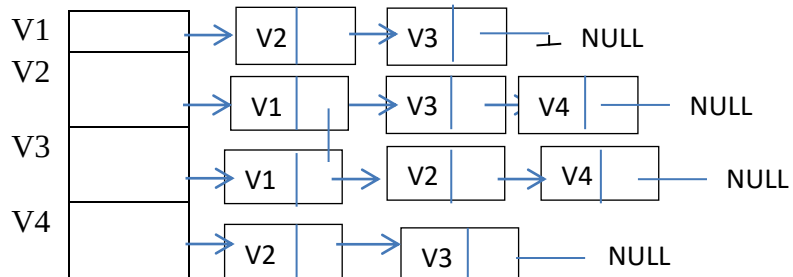
2. Adjacency list:

An adjacency list represents a graph as an linked structure .We store all vertices in list, and each vertex have a linked list of its adjacency vertices.

Example:



Adjacency List Representation is:



3.Incidence Matrix:

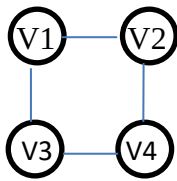
An incidence matrix is a matrix where:

- Rows represent **vertices**
- Columns represent **edges**
- Entries show whether a vertex is incident (connected) to an edge

For an undirected graph:

- If vertex v is connected to edge $e \rightarrow \text{value} = 1$
- Otherwise $\rightarrow \text{value} = 0$

Example:

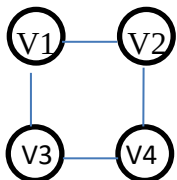


Vertex/edges	$e1(v1-v2)$	$e2(v2-v4)$	$e3(v4-v3)$	$e4(v3-v1)$
V1	1	0	0	1
V2	1	1	0	0
V3	0	0	1	1
V4	0	1	1	0

4.Edge List:

An edge list is a graph representation in which each edge is stored as a pair of vertices (u, v) . If the graph is weighted, each edge may also include its weight.

Example:



Here $(v1,v2)=(v2,v1)$,defined as one edge list.

If it is directed, represent as $(u \rightarrow v)$

If it is undirected, represent as (u,v)

If it is weighted graph, represent as $(u,v,height)$

Edge List Representation

$(V1, V2)$

$(V2, V4)$

(V4, V3)
(V3, V1)

APPLICATIONS OF GRAPHS

1. Computer Networks

- Computers are represented as **nodes**
- Connections (LAN, internet links) are **edges**
 - Used in routing and network design

2. Social Networks

- Users = nodes
- Friend/follow relationships = edges
 - Example: Facebook, Instagram, LinkedIn

3. Google Maps / Navigation

- Cities or locations = nodes
- Roads = edges
 - Used to find **shortest path** between places

4. Web Pages (Internet)

- Web pages = nodes
- Hyperlinks = edges
 - Used in search engines like Google

5. Airline and Railway Networks

- Airports/stations = nodes
- Flights/trains = edges
 - Used for scheduling and route optimization

6. Project Scheduling

- Tasks = nodes
- Dependencies = edges
 - Used in project planning (PERT/CPM)

7. Recommendation Systems

- Users and products = nodes
- Preferences = edges
 - Used in YouTube, Netflix, Amazon suggestions

8. Biology

- Atoms, molecules, or proteins = nodes
- Interactions = edges
 - Used in studying biological networks